

# Final Project Proposal

## *Telco Customer Churn Prediction*

### 1. Certificate Name

Mohamed Ali Shoble

---

### 2. Project Title and Description

**Title:** Telco Customer Churn Prediction

Customer churn — when a subscriber cancels their service — is one of the most expensive problems telecom companies face, since acquiring a new customer typically costs far more than retaining an existing one. This project builds a machine learning model that predicts, from a customer's account and service details, whether they are likely to churn. The final deliverable is a REST API that accepts customer data as JSON and returns a churn prediction with a probability score — useful for a retention team to proactively target at-risk customers before they cancel.

---

### 3. Problem Type

Classification — binary output: Churn (Yes) or No Churn (No).

The target column is Churn. This is supervised learning: we train on historical customers where the outcome (whether they left) is already known.

---

### 4. Dataset

- Source: IBM Telco Customer Churn Dataset (Kaggle) — [kaggle.com/datasets/blattnar/telco-customer-churn](https://kaggle.com/datasets/blattnar/telco-customer-churn)
- Size: 7,043 rows, 21 columns
- Target: Churn — whether the customer left within the last month (Yes/No)

**Main features:**

- tenure — number of months the customer has stayed (numeric)
- MonthlyCharges, TotalCharges — billing amounts (numeric)
- Contract — month-to-month, one year, two year (categorical)
- PaymentMethod, PaperlessBilling — billing behavior (categorical)
- InternetService, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies, PhoneService, MultipleLines — services subscribed to (categorical)
- gender, SeniorCitizen, Partner, Dependents — demographics (categorical/binary)

Preprocessing plan: handle missing/blank TotalCharges values, encode categoricals (one-hot or label encoding), scale numeric features, stratified train/test split (80/20) to preserve the churn/no-churn ratio.

---

## 5. Algorithms I Plan to Train

| # | Algorithm                   | Why it fits                                                                                                                                                  |
|---|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Logistic Regression         | Bootcamp baseline for binary classification; fast, interpretable coefficients show which features drive churn.                                               |
| 2 | Decision Tree               | Bootcamp model; captures non-linear splits (e.g. contract type × tenure) and is easy to explain to non-technical stakeholders.                               |
| 3 | Random Forest               | Bootcamp ensemble; reduces the overfitting a single Decision Tree is prone to.                                                                               |
| 4 | XGBoost (Gradient Boosting) | Researched — handles mixed categorical/numeric features and mild class imbalance well; typically, the top performer on this dataset in published benchmarks. |

This exceeds the minimum of three algorithms. Three (Logistic Regression, Decision Tree, Random Forest) come from the bootcamp. XGBoost I will research via scikit-learn/XGBoost documentation and one tutorial.

---

## 6. Evaluation Plan

**Metrics for all models (on the same held-out test set):**

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix

Best-model rule: Rank all trained models by F1-Score on the test set. The dataset is imbalanced (~73% no-churn vs ~27% churn), so plain Accuracy would be misleading — a model that predicts "no churn" for everyone would score ~73% while being useless. F1 balances Precision and Recall, which matters here because both false positives (wasting a retention offer on a loyal customer) and false negatives (missing a customer about to leave) are costly. If two models tie on F1, I will break the tie with higher Recall (catch more actual churners).

I will print a comparison table with one row per algorithm and save/deploy only the winner.

---

## 7. Deployment Sketch

- Framework: FastAPI (simple, modern, auto-generates API docs at /docs)
- Endpoint: POST /predict

### Input JSON example:

```
{
  "tenure": 12,
  "MonthlyCharges": 70.35,
  "TotalCharges": 845.50,
  "Contract": "Month-to-month",
  "PaymentMethod": "Electronic check",
  "PaperlessBilling": "Yes",
  "InternetService": "Fiber optic",
  "OnlineSecurity": "No",
  "TechSupport": "No",
  "gender": "Female",
  "SeniorCitizen": 0,
  "Partner": "No",
  "Dependents": "No"
}
```

### Output JSON example:

```
{
  "prediction": "Churn",
  "probability": 0.82
}
```

The API loads the best saved model from `models/best_model.pkl` plus any preprocessing artifacts (scaler, encoders).

---

## 8. Repository Plan

```
telco-churn-prediction/
├── dataset/
│   └── telco_customer_churn.csv
├── src/
│   ├── preprocess.py    # cleaning, encoding, scaling, train/test split
│   └── train.py         # train all models, compare, save best
├── api/
│   └── app.py           # FastAPI app with /predict
├── models/
│   ├── best_model.pkl
│   └── scaler.pkl
├── notebooks/
│   └── exploration.ipynb # optional EDA
├── README.md
├── requirements.txt
└── project_paper.md
```

### Run commands (planned):

```
python src/train.py      # trains models, prints comparison table, saves best
uvicorn api.app:app --reload # starts API locally
```